



MANAGEMENT DEVELOPMENT REPORT

Streaming Administration Portal

A consolidated backend control centre for video streaming, audio rooms, users/senders, hosts, agencies, digital assets, security and operational oversight.

Reporting period: 14 July 2026 – 27 July 2026

Prepared for: Management Review

Project: Streamline Administration Portal

Document date: 27 July 2026

Status: Active development / functional backend foundation

1. Executive Summary

The Streamline Administration Portal was started as a new administrative backend for a combined video and audio streaming application, comparable in operating model to platforms such as StreamKar and OyeTalk. The mobile application is being developed separately in React Native CLI; this portal provides the operational interface, persistent data layer, mobile APIs and real-time controls required by administrators.

The work progressed from an initial administrator login screen into a multi-module system covering users/senders, hosts, audio rooms, agencies, content uploads, bans, devices, virtual currency, special IDs and audit logs. PostgreSQL and Prisma now store the principal records, NextAuth protects administrative access, and Socket.IO communicates immediate account and room events to the mobile application.

At the end of this reporting period, the portal has a working technical foundation and several operationally functional modules. Some business modules—particularly the deeper agency workflows, live-stream operations and reporting—remain UI-ready or planned and require final rules, integrations and production data.

23 Recorded development commits	20 Prisma/PostgreSQL models	10 API route files	8 Upload/content categories
---	---	--	---

Primary outcomes

- A responsive Tailwind-based administration interface with dashboard navigation and global feature search.
- Database-backed user, host, device, transaction, ban, room, special-ID, upload and audit records.
- Mobile registration, login, profile, session-status, audio-room and downloadable asset APIs.
- Real-time Socket.IO events for bans, forced logout, device restrictions, room seats and room control.
- Time-based controls that automatically expire bans, room restrictions, special IDs and uploaded-asset grants.

2. Project Scope and Technical Foundation

Area	Implementation	Purpose
Web framework	Next.js 16 App Router, React 19	Server-rendered portal pages, client-side management interfaces and route handlers.
Design system	Tailwind CSS 4	Responsive portal UI without page-specific custom CSS.
Database	PostgreSQL with Prisma 7.8 and PostgreSQL adapter	Persistent operational records, relationships, indexes and transactional updates.
Admin authentication	NextAuth	Protected administrative pages and actions.
Mobile authentication	Bearer session tokens with session-version checks	Registration, login and protected mobile endpoints.
Real-time communication	Custom Node server and Socket.IO	Immediate security events, room updates and seat coordination.
Media storage	Database-backed upload assets with signed file URLs	Controlled delivery of images, GIFs, WebP and supported video assets to the app.

Core database coverage

The schema contains 20 models, including administrators, audit logs, users, hosts, devices, gifts, coin adjustments, bans, verification, salaries, albums, special IDs, game logs, live sessions, performance, violations, audio rooms and upload assignments. Relationships and indexes were added for common searches such as user status, VIP level, device MAC address, ban expiry, room state and assignment expiry.

Security approach

- Passwords are stored as hashes rather than recoverable plain text.
- Administrators can reset a user password and receive a generated temporary password; the reset increments the session version and forces active sessions to log out.
- Soft deletion preserves operational history while removing login capability and personal account identifiers.
- Administrative actions are recorded with administrator, entity, reason, metadata and timestamp.
- Signed upload URLs are bound to the user and session version and expire after a limited period.

3. Delivered Portal Features

Module	Delivered functionality	Status
Admin access	Admin login, protected pages, sign-out and initial authenticated layout.	Functional
Dashboard	Platform summary cards, activity presentation, management shortcuts and global navigation search.	Functional
Users / Senders	User list, phone-first identity, VIP filter, add user, roles, five VIP levels, special IDs, password reset, soft deletion and clickable detail pages.	Functional
User finance	Gift/spending history, total spending, balance, total top-up and audited add/remove coin adjustments.	Functional
User supporting records	Levels, albums, top history, games, device information and audio-room records.	Mixed: functional + seeded/UI data
Host management	Host list, filters, add host, profile pages, gifts received, salary history, devices, verification, live history, performance and violations.	Core structure functional
Security controls	User/device ban, unban, proof attachment reference, predefined/custom duration, force logout and automatic expiry reconciliation.	Functional
Audit logs	Portal-wide administrative activity records with search by administrator, user/entity ID or activity.	Functional
Special IDs	4–7 character definitions, VIP/top-up eligibility, admin assignment, 3/7/15/30-day or custom duration, automatic expiry and normal-ID restoration.	Functional
Audio rooms	Persistent owner-bound room IDs, room discovery, room moderation, timed joining/block/termination controls and real-time seat relay.	Functional foundation
Agency management	Agency home, ranking, top hosts, total agency recharge, tasks, monthly salary, host salary and applications tabs.	UI/prototype data
Uploads	Banners, frames, entrances, tail-lights, gifts, badges, chat boxes and room backgrounds, including preview, details, tags and user grants.	Functional

Automatic refresh

A portal-wide refresh component was added so administrators no longer need to manually reload pages to see newly created or modified database records. Refresh also occurs when the browser regains focus or visibility.

4. Mobile Application Integration

The backend was designed to support the separately developed React Native CLI application. Current mobile-facing endpoints and real-time behaviours include:

Integration	Capability
User registration	Phone number required and unique; email optional but unique when supplied; password hashing; optional profile attributes.
User login	Phone/password login, session token issuance, IP/MAC/location/device capture, user-ban and device-ban enforcement.
Profile update	Name, contact details, country, profile image, gender and date of birth; gender and DOB are locked after first set.
Session status	Session version, forced logout, account ban, device ban, special ID, gender and DOB status.
Audio rooms	Create/reopen the owner's persistent room ID, retrieve own room, discover other live rooms and apply moderation restrictions.
Room discovery	Returns up to 50 live, joinable, unblocked rooms excluding the caller's own room.
Upload catalogue	Returns global assets and assets actively assigned to the caller, with signed URLs suitable for React Native media rendering.

Real-time Socket.IO events

- Immediate account ban/unban and device ban/unban notification.
- Forced logout after administrative action, password reset or account deletion.
- Special-ID assignment, revocation and automatic-expiry notification.
- Audio-room join/leave, participant updates and automatic resource release when a room becomes empty.
- Owner-authorized seat updates, seat requests and accept/reject responses; requester name and profile image are included.
- Timed room-control expiry events for joining, block and termination restrictions.

5. Key Business Workflows

User lifecycle

1. A user registers through the mobile API using a unique phone number; email is optional and cannot be duplicated.
2. Login validates credentials, account status and the current device before issuing a session.
3. Administrators can manage roles, VIP status, coins, special IDs, bans, password reset and account deletion.
4. Every sensitive administrative action is written to the audit log and relevant real-time events are sent to the app.

Time-based control model

A common minute-based duration system is used throughout the portal. Administrators can select predefined periods or enter custom minutes, while the UI displays the calculated human-readable value. The approach is used for bans, special IDs, room controls and uploaded-asset grants.

Upload and entitlement workflow

1. An administrator uploads an image, GIF, WebP or supported video into one of eight content categories.
2. The asset stores a title, details, tags, file metadata, media bytes and room-background eligibility.
3. The asset may be global or granted to multiple users. Each user grant has an independent duration and expiry.
4. Only global or actively granted assets appear in the mobile catalogue. Expired access is also rejected by the file endpoint.
5. Assigned items are displayed in the relevant user's dedicated portal profile.

Audio-room lifecycle

1. The system assigns a persistent room ID to a room owner.
2. When the room becomes empty, live resources are released and the room becomes idle, but its ID remains attached to the owner.
3. When the owner starts again, the same ID is reactivated.
4. Administrators can disable joining, block or terminate the room for a set time, then manually restore it or allow automatic expiry.

6. Issues Encountered and Resolutions

Issue	Impact	Resolution
Next.js initially searched for a legacy pages directory.	Development server could not start with the App Router structure.	Project configuration/dependencies were aligned with the App Router and current Next.js conventions.
Prisma schema was initially missing and database authentication/sync failed.	No reliable persistent data layer.	A complete PostgreSQL schema and Prisma configuration were created, then pushed and generated successfully.
UI dummy-data assumptions conflicted with database types (VIP values, null devices and missing exports).	Runtime crashes on user and host pages.	Data mapping was normalized, null handling added and hard-linked data moved into database-backed structures or safe seed data.
New fields such as audit logs, deleted account state and supporting relations were queried before the client/schema matched.	Prisma validation errors including unknown fields.	Schema updates, client regeneration, startup generation and runtime schema-version checks were introduced.
Force logout queried fields absent from the generated Prisma client.	Administrative force logout returned a server error.	sessionVersion and forcedLogoutAt were added and session validation was wired end-to-end.
Timed bans did not release users automatically.	Users remained blocked after the selected duration.	Active-ban queries, expiry reconciliation and scheduled Socket.IO unban events were implemented.
Device bans were stored but not enforced by login/status routes.	Banned devices could continue or re-establish access.	Device lookup, login rejection, session-status fields and immediate device socket events were added.
Room records were deleted or lost when the host left.	Owners did not retain a stable room identity.	Owner-bound unique room IDs and an IDLE resource-release lifecycle were implemented.
Assigned room backgrounds appeared white in the mobile app.	App received a value but could not render the media reliably.	Database media delivery, signed absolute URLs, correct MIME responses and LAN/external-origin handling were implemented.
Room discovery returned 401 or did not expose suitable rooms.	Trending Parties could not load.	Bearer-token validation and a dedicated filtered discover endpoint were completed.
Special IDs and upload grants did not expire safely.	Temporary benefits could remain active or accidentally become global.	Explicit expiry fields, active-only queries, scheduled/reconciled expiry and a separate global-asset flag were added.
Prisma Fast Refresh reused a client that lacked new assignment fields.	User profile failed on expiresAt.	The cached client now validates required model fields and replaces stale instances when the schema version changes.

7. Development Timeline

- **14 July — Project foundation**
Next.js project initialization and initial portal direction.
- **15–16 July — User/host management and audit foundation**
Management pages, profiles, bans, gift history, initial database/API configuration and audit logs.
- **17 July — Sessions and real-time server**
Session versioning, force-logout foundation, Socket.IO installation and expanded management tabs.
- **20 July — Audio rooms and Special IDs**
Room moderation records, user-management improvements and temporary Special-ID model.
- **21 July — Login and persistent rooms**
Mobile login fixes, device/security improvements, room ID persistence and dedicated user profile pages.
- **22 July — Room controls and agencies**
Audio-room restriction refinements, agency management UI and mobile/backend coordination updates.
- **23 July — Room lifecycle refinement**
Resource release while retaining persistent owner room IDs.
- **24 July — Upload platform and onboarding data**
Upload categories, database media storage, mobile catalogue, gender/DOB support and background delivery fixes.
- **25–27 July — Multi-user and timed entitlements**
Asset detail management, multiple recipients, per-user expiry, safe access control and Prisma cache hardening.

8. Current Status, Gaps and Risks

Operationally functional

- Admin authentication, navigation and global feature search.
- Core user records, mobile registration/login/profile/session APIs and device capture.
- Security actions, automated expiry, force logout and audit logging.
- Persistent audio-room identity, discovery and Socket.IO coordination.
- Database-backed upload catalogue, signed media delivery and timed multi-user entitlements.

UI-ready or partially populated

- Several supporting user/host tables use representative or seed data until the mobile application sends all corresponding records.
- Agency ranking, tasks, salary and application tabs are designed but require final database entities, business rules and workflows.
- Audio recording and permanent room deletion code are intentionally not exposed in the interface.
- Live-stream management and formal reporting navigation are placeholders for a later phase.

Risks requiring management decisions

Infrastructure: Media is currently stored inside PostgreSQL. This is suitable for controlled development and early testing, but object storage/CDN should be adopted before high-volume production use.

Mobile media origins: Physical devices cannot use the backend's localhost address. Production requires a stable HTTPS API/media hostname and configured mobile CORS origin.

Permissions: The portal currently has administrator authentication, but granular role-based access control should be finalized for Super Admin, finance, moderation, content and support teams.

Testing: Lint and production builds pass, but automated integration, end-to-end, load and security testing should be added before production launch.

9. Recommended Next Phase

- 1. **Finalize business requirements:** confirm agency commission formulas, host salary calculations, moderation policies, reporting KPIs and admin permission levels.
- 2. **Complete remaining database modules:** convert remaining representative tables and agency workflows to live database records and mobile event ingestion.
- 3. **Production media architecture:** move uploaded media to S3-compatible object storage with CDN delivery, virus/type validation and image/video processing.
- 4. **Production security:** add granular RBAC, rate limiting, login throttling, secret rotation, HTTPS-only cookies/tokens and formal audit retention.
- 5. **Quality assurance:** create API contract tests, database integration tests, Socket.IO tests and browser/mobile end-to-end scenarios.
- 6. **Deployment and monitoring:** establish staging/production environments, managed PostgreSQL, backups, error monitoring, uptime alerts and structured logs.
- 7. **Mobile integration sign-off:** jointly validate registration, login, bans, force logout, room discovery, seats and assigned media on physical Android/iOS devices.

Management conclusion

The portal has moved well beyond a visual prototype. It now provides a connected administrative foundation with persistent data, secure session control, mobile APIs and real-time operational actions. The most important next step is to convert the remaining business placeholders into approved production workflows, strengthen infrastructure and security, and execute formal end-to-end testing with the React Native application.

Prepared by / Development Team	Reviewed by / Management
--------------------------------	--------------------------

This report reflects the repository and implemented functionality available as of 27 July 2026. Status labels distinguish fully wired features from UI-ready or planned work.